

Yes — for **Playwright with TypeScript**, Log4j is **not normally used** because Log4j is a Java logging framework. In your Playwright TypeScript framework, the equivalent approach is to use a Node.js/TypeScript logging library such as **Winston** or **Pino**.

For your Playwright + TypeScript automation, **Winston is a good training-friendly choice**.

1. Install Winston

Bash

```
npm install winston
```

2. Create `utils/logger.ts`

TypeScript

```
import winston from 'winston';

const logger = winston.createLogger({
  level: 'info',

  format: winston.format.combine(
    winston.format.timestamp({
      format: 'YYYY-MM-DD HH:mm:ss'
    }),
    winston.format.printf(({ timestamp, level, message }) => {
      return `${timestamp} ${level.toUpperCase()} - ${message}`;
    })
  ),

  transports: [

    // Console
    new winston.transports.Console(),

    // Log file
    new winston.transports.File({
      filename: 'logs/automation.log'
    })
  ]
});

export default logger;
```

Project structure:

```
Typescript
|
├─ tests
│   └─ login.spec.ts
|
├─ utils
│   └─ logger.ts
|
├─ logs
│   └─ automation.log
|
├─ playwright.config.ts
└─ package.json
```

3. Use it in a Playwright test

TypeScript

```
import { test, expect } from '@playwright/test';
import logger from '../utils/logger';

test('Valid Login Test', async ({ page }) => {

    logger.info('Test started');

    logger.info('Opening application');

    await page.goto('https://example.com');

    logger.info('Application opened successfully');

    logger.info('Checking page title');

    await expect(page).toHaveTitle(/Example/);

    logger.info('Page title validation successful');

    logger.info('Test completed');

});
```

You will see:

```
2026-09-22 10:05:12 info - Test started
2026-09-22 10:05:12 info - Opening application
2026-09-22 10:05:13 info - Application opened successfully
2026-09-22 10:05:13 info - Checking page title
2026-09-22 10:05:13 info - Page title validation successful
2026-09-22 10:05:13 info - Test completed
```

And the same information will be stored in:

```
logs/automation.log
```

4. Log errors

This is particularly useful with Playwright:

TypeScript

```
try {

  logger.info('Opening login page');

  await page.goto('https://example.com/login');

  logger.info('Login page opened');

} catch (error) {

  logger.error(`Failed to open login page: ${error}`);

  throw error;

}
```

5. Logging in a Page Object

For your Playwright POM framework, this is very useful.

TypeScript

```
import { Page, Locator } from '@playwright/test';
import logger from '../utils/logger';

export class LoginPage {

  readonly page: Page;
  readonly username: Locator;
  readonly password: Locator;
  readonly loginButton: Locator;

  constructor(page: Page) {

    this.page = page;

    this.username = page.locator('#username');
    this.password = page.locator('#password');
    this.loginButton = page.locator('#login');

  }

}
```

```

    async login(username: string, password: string) {

        logger.info('Entering username');

        await this.username.fill(username);

        logger.info('Entering password');

        await this.password.fill(password);

        logger.info('Clicking Login button');

        await this.loginButton.click();

        logger.info('Login operation completed');
    }
}

```

Then:

TypeScript

```

test('Login Test', async ({ page }) => {

    const loginPage = new LoginPage(page);

    logger.info('Starting login test');

    await page.goto('https://example.com/login');

    await loginPage.login(
        'admin',
        'Password123'
    );

    logger.info('Login test completed');
});

```

6. Don't log passwords

Avoid:

TypeScript

```

logger.info(`Username: ${username}`);
logger.info(`Password: ${password}`);

```

Instead:

TypeScript

```
logger.info(`Logging in user: ${username}`);
```

7. Playwright + Winston + Allure

Since you're already working with Playwright reporting, you can use:

```
Playwright
├── Winston
│   └── automation.log
├── HTML Reporter
├── Allure
└── Trace
```

They serve different purposes:

Tool	Purpose
Winston	Execution/application logs
Playwright HTML	Test results
Allure	Rich test reporting
Trace Viewer	Detailed failure/debugging
<code>console.log()</code>	Quick temporary debugging

Key point: If your automation is **Playwright + TypeScript**, don't try to bring Log4j into the project. Use **Winston/Pino** for the equivalent logging functionality.

If you want a framework-style implementation, the next useful step is a **Playwright + TypeScript Logger utility with INFO/DEBUG/WARN/ERROR, daily rolling log files, screenshots on failure and TestNG-listener-equivalent Playwright hooks.**



SerpApi, LLC

Ad

SerpApi: Google Search API

Access Google Search, Maps, Shopping, and other search engine with a simple API